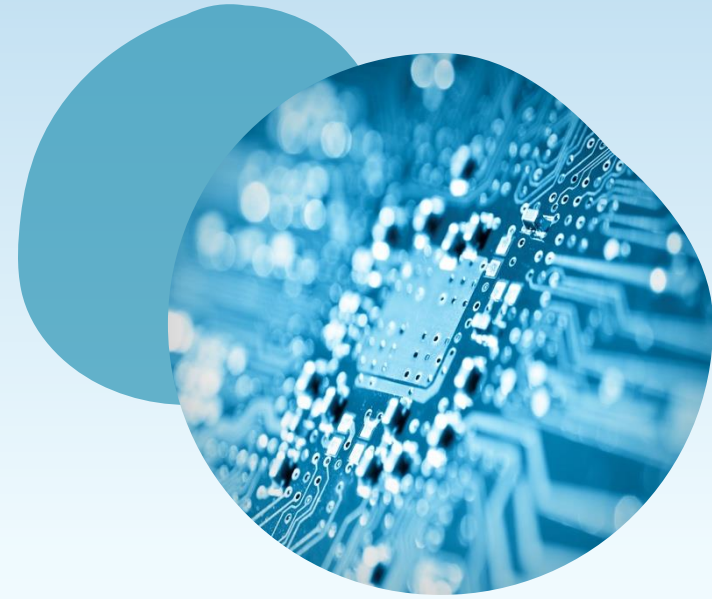




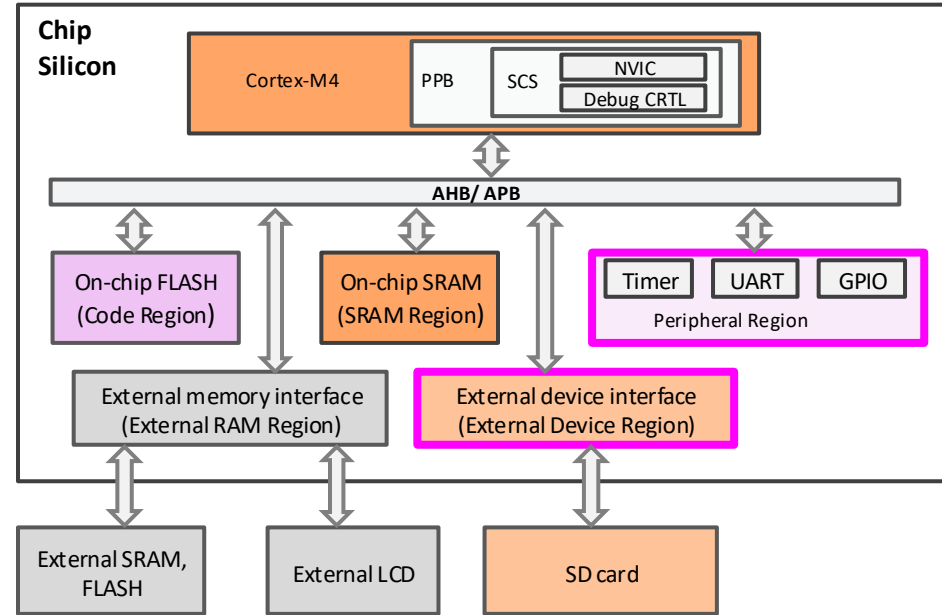
Processors and I/O

Embedded Software for the Internet of Things

Kasım Sinan Yıldırım – University of Trento

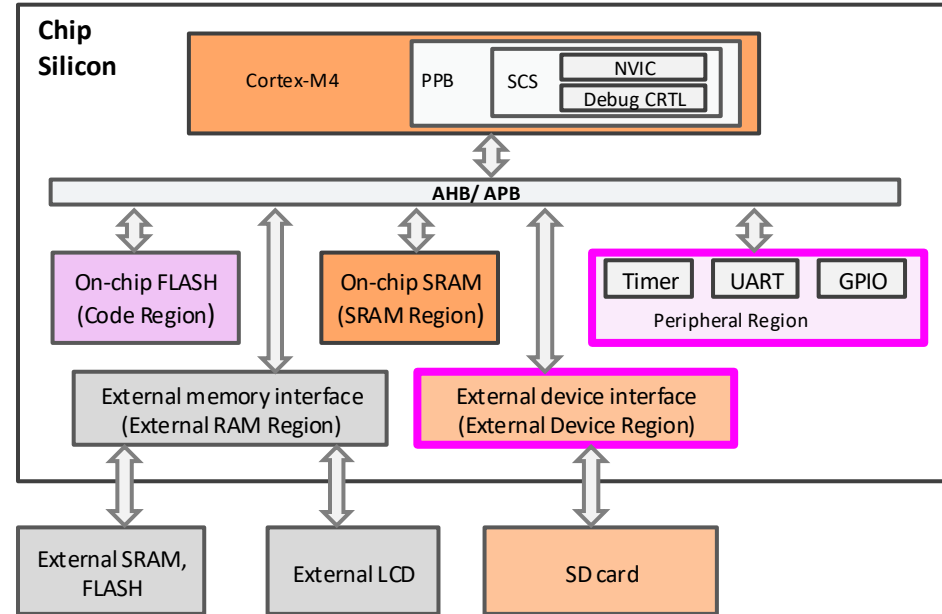
Processor and Input and Output Devices

- Other than the CPU, we have many peripherals (e.g., I/O devices)
 - Timers, UART, Sensors



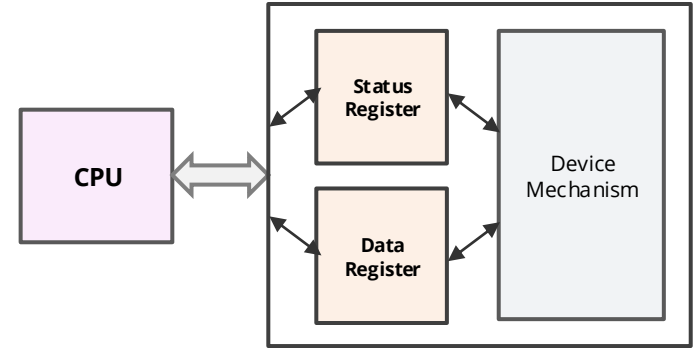
How Do They Communicate?

- The processor reaches **on-chip** peripherals through the **bus matrix**
- External interfaces connect **off-chip** devices



How Do They Communicate?

- Each peripheral (i.e., device) exposes registers
 - The CPU talks to the device by reading and writing **device registers**.
- We will later see that interrupt lines (not shown in this figure) will report events from I/O devices that need CPU service



How Do They Communicate?

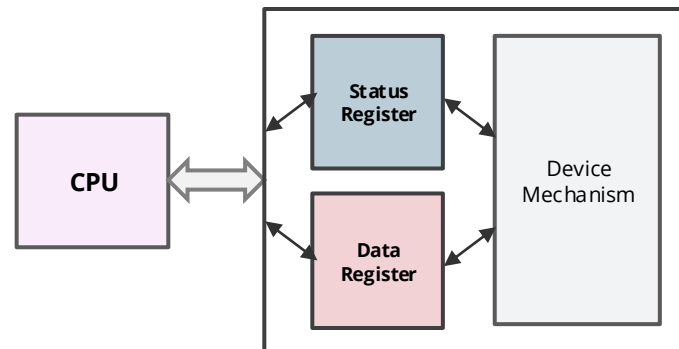
- Device registers connect software to **hardware state**

■ Data registers

- Carry values moving into or out of the device.
 - hold data values
- Access direction depends on the peripheral.
- Can be readable or writable

■ Status and control registers

- Report events and configure operating modes.
 - e.g., event: the current operation has been completed
- Individual bits often have different meanings.



Two architectural models for device access

- Two ways to access those registers.
 - **I/O instructions**
 - special instructions for input and output
 - **in** and **out** in the case of the Intel x86
 - provide a separate address space for I/O devices (separate address lines...)
 - **Memory-mapped I/O**
 - Provides addresses for the registers in each I/O device
 - programs use the CPU's normal memory read and write instructions to communicate with the devices.

Model	Addressing	Instructions	Typical example
Separate I/O space	Dedicated device address space	Special input and output instructions	Legacy x86 ports
Memory-mapped I/O	Device registers share the memory address space	Normal load and store instructions	Arm microcontrollers

ARM loads and stores access device registers

- Define location for device and provide Read/Write code:

```
DEV1_BASE EQU 0x40001000

LDR  r1, = DEV1_BASE
LDR  r0, [r1]          // read register
MOVS r0, #8
STR  r0, [r1]          // write register
```

What matters

The address must identify the correct register. The access width and alignment must match the peripheral specification.

A wrong address can control a different peripheral or trigger a bus fault.

C requires **volatile** access to device registers

- We can use **pointers** to manipulate the addresses of I/O devices.

```
#include <stdint.h>

#define DEV1_STATUS_ADDR 0x40001000u
#define DEV1_STATUS (*(volatile uint32_t *)DEV1_STATUS_ADDR)

uint32_t status = DEV1_STATUS;
DEV1_STATUS = 8u;
```

volatile

Tells the compiler that every read and write matters because hardware may change the value outside normal program flow.

C requires **volatile** access to device registers

- Introduce functions that read and write memory locations:

```
#define DEV1_STATUS_ADDR 0x40001000u

int read(uint8_t *location) {
    return *location;
}

void write(uint8_t *location, char newval) {
    (*location) = newval;
}

uint32_t status = read(DEV1_STATUS_ADDR); /* read device register */
write(DEV1_STATUS_ADDR, 8); /* write 8 to device register */
```

Register definitions should express width and intent

- The device reference manual defines the legal access pattern for each device register.

Concern	Reason
Access width	A byte, halfword, or word transfer may have different hardware meaning
Read-only and write-only behavior	Some registers reject unsupported access directions
Reserved bits	Software should preserve or write required values
Side effects	A read may clear status and a write may acknowledge an event

Polling (Busy-Wait) I/O

- I/O devices are typically slower than the CPU
 - They may require many cycles to complete an operation.
- **Busy-wait** (Often called **polling**)
 - Asking an I/O device whether it is finished by reading its **status register**
 - Software reads a status register until the ready or complete condition appears
- The loop provides low control-flow complexity
 - CPU cycles are consumed while no useful application work progresses

Polling an output device (Example)

- One character starts each transaction
- The inner loop waits for completion

```
#define OUT_CHAR 0x1000 /* output device character register */
#define OUT_STATUS 0x1001 /* output device status register */

while (*current != '\0') {
    OUT_DATA = *current;
    OUT_STATUS = START;

    while (OUT_STATUS != DONE) {
        // CPU waits here
    }

    current++;
}
```

The CPU cannot perform
independent application work
during the wait

A polling-based input-to-output loop (Example)

- Repeat forever
 - Wait until the input status reports available data
 - Read one value from the input data register
 - Write that value into the output data register
 - Wait until the output device completes the transaction
- The loop does not allow the **foreground work** to continue.

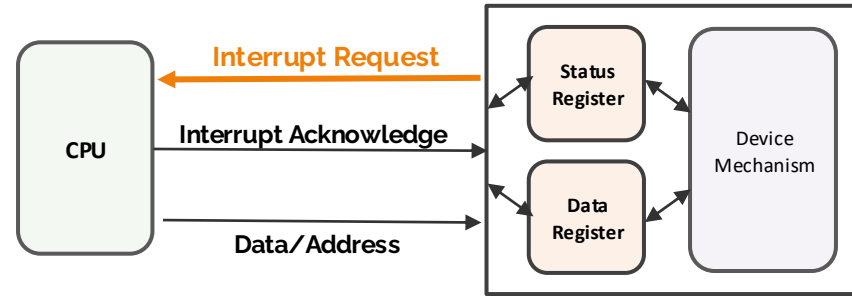
```
for (;;) {  
    while (IN_STATUS == EMPTY) { }  
    uint8_t c = IN_DATA;  
  
    while (OUT_STATUS == BUSY) { }  
    OUT_DATA = c;  
    OUT_STATUS = START;  
}
```

Interrupt-driven I/O

- Busy-wait I/O is **inefficient**.
 - The CPU could do useful work in parallel with the I/O
- The interrupt mechanism allows a device requests service only when an **event** occurs.
- The processor **saves context**, runs a **handler**, then **resumes** the interrupted program.

Several signals control the interrupt process

- The device asserts an **interrupt request**
 - The I/O device's logic decides when to interrupt
 - e.g., when data is ready (user pressed a button)
- The CPU accepts the request
 - It asserts the **interrupt acknowledge** signal
 - It changes the program counter to point to the device's **interrupt handler**
- The **interrupt handler** identifies the event source (e.g., which device) and takes the necessary processing actions.



Interrupt entry changes control flow

- When an interrupt occurs:
 - First, the value of the PC at the interruption is saved:
 - the CPU can return to the **foreground program** later
 - Then, the PC starts pointing to an **interrupt handler** routine:
 - this routine takes care of the device by reading data that have just become ready, writing the next data...

Phase	Processor action
Recognition	Detect interrupt
Context save	Preserve the return state and selected registers
Vector fetch	Load the handler address from the vector table
Service	Execute the interrupt handler
Return	Restore state and resume the interrupted code

Cortex-M interrupt entry

- We will see details later.
- Save the necessary context
 - Pushes important registers R0 to R3, R12, LR, PC, and xPSR into stack
 - Finds the handler address from the interrupt vector table
 - The handler may save additional registers that it uses
 - Interrupt return restores the hardware-saved state and context
- Interrupt code in C is processed in a specific way by the compiler:
 - Compiler-generated handlers follow the architecture calling convention when declared correctly.

A minimal input interrupt handler in C

- Read the data that caused the interrupt
- Record minimal state for later processing
- Acknowledge or clear the device event
- Return quickly

```
volatile uint8_t latest_char;  
volatile bool char_ready;  
  
void input_handler(void) {  
    latest_char = IN_DATA; // read device register  
    char_ready = true;  
    IN_STATUS = ACK; // acknowledge the device  
}
```

Interrupts (Example)

- Copying Characters From Input to Output With Basic Interrupts
 - **achar**: passes the character to the foreground program.
 - **gotchar**: signals when a new character has been received.

INTERRUPT HANDLERS

```
/* get a character and put in global (called when IN_STATUS is 1) */
void input_handler() {
    achar = read(IN_DATA); /* get character */
    gotchar = TRUE;        /* signal to main program */
    write(IN_STATUS,0);    /* reset status to initiate next transfer */
}

/* react to character being sent (called when OUT_STATUS is 0) */
void output_handler() {
    /* don't have to do anything */
}
```

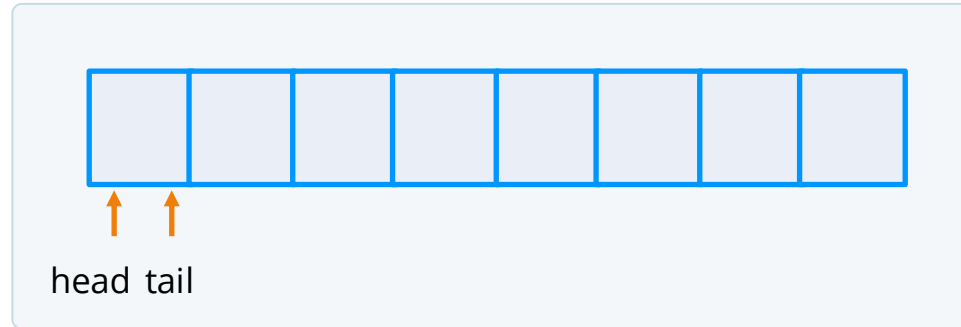
Interrupts (Example)

- Copying Characters From Input to Output With Basic Interrupts
 - The main program somewhat simpler compared to Busy Wait I/O
 - But still does not let the foreground program do useful work.
 - It **polls** input and writes data...

```
main() {  
    while (TRUE) { /* read then write forever */  
        if (gotchar){ /* write a character */  
            write(OUT_DATA,achar); /* put character in device */  
            write(OUT_STATUS,1);    /* set status to initiate write */  
            gotchar = FALSE;        /* reset flag */  
        }  
    }  
}
```

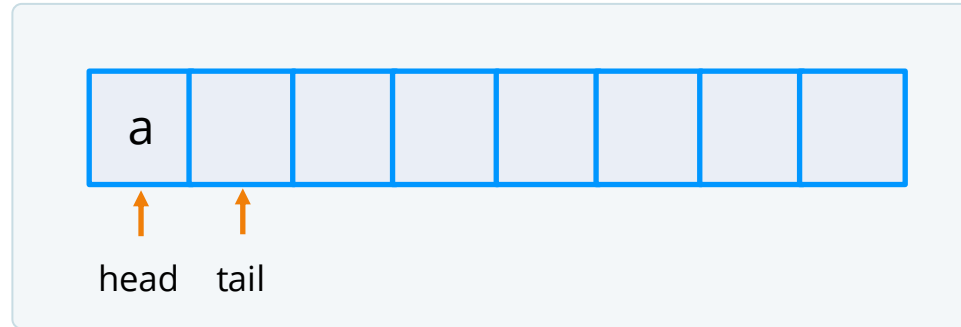
A circular buffer separates producer and consumer timing

- One index identifies the next read position.
- The other identifies the next write position.



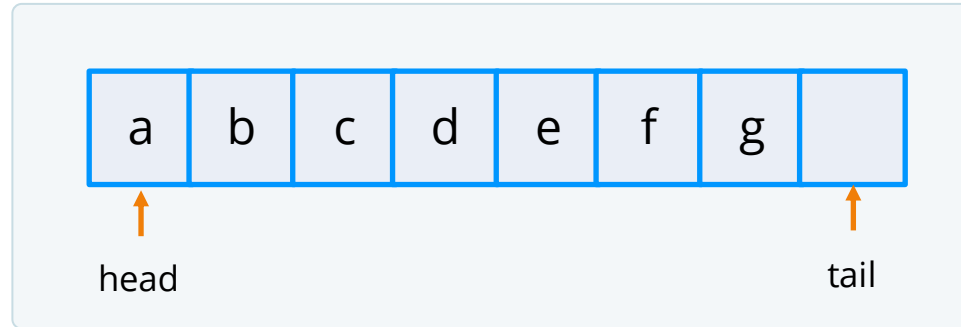
A circular buffer separates producer and consumer timing

- One index identifies the next read position.
- The other identifies the next write position.



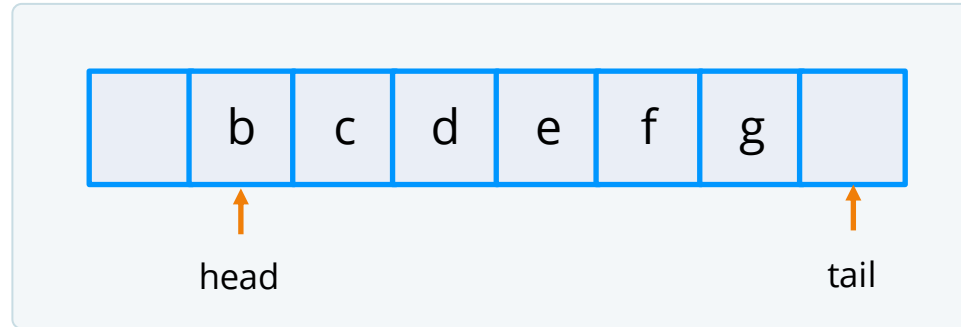
A circular buffer separates producer and consumer timing

- One index identifies the next read position.
- The other identifies the next write position.



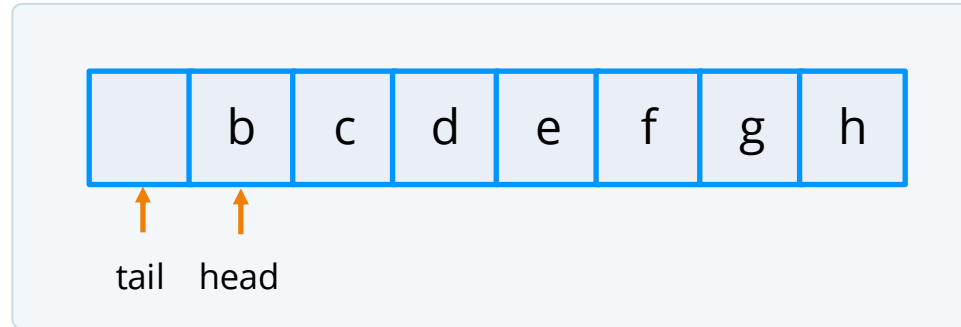
A circular buffer separates producer and consumer timing

- One index identifies the next read position.
- The other identifies the next write position.



A circular buffer separates producer and consumer timing

- One index identifies the next read position.
- The other identifies the next write position.



Interrupts (Example) - With Circular Buffer

```
#define BUF_SIZE 8
char io_buf[BUF_SIZE]; /* character buffer */
int buf_head = 0, buf_tail = 0; /* current position in buffer */
int error = 0; /* set to 1 if buffer ever overflows */

int buffer_empty() { /* returns TRUE if buffer is empty */
    return buf_head == buf_tail;
}

int buffer_full() { /* returns TRUE if buffer is full */
    return (buf_tail+1) % BUF_SIZE == buf_head ;
}

int nchars() { /* returns the number of characters in the buffer */
    if (buf_head >= buf_tail)
        return buf_head - buf_tail;
    else
        return BUF_SIZE - buf_tail - buf_head;
}
```

Interrupts (Example) - With Circular Buffer

```
void buffer_put(char achar) { /* add a character to the buffer head */
    io_buf[buf_tail++] = achar;
    /* check pointer */
    if (buf_tail == BUF_SIZE)
        buf_tail = 0;
}

char buffer_get() { /* take a character from the buffer head */
    char achar;
    achar = io_buf[buf_head++];
    /* check pointer */
    if (buf_head == BUF_SIZE)
        buf_head = 0;

    return achar;
}
```

Input ISR adds data and starts output

- Capture the arriving item
- Record overflow without blocking
- Kick the output only when the queue was idle
- Acknowledge the input event

```
void input_handler(void) {  
    uint8_t c = IN_DATA;  
  
    if (buffer_full()) {  
        overflow = true;  
    } else {  
        bool was_empty = buffer_empty();  
        buffer_put(c);  
        if (was_empty) start_output();  
    }  
  
    IN_STATUS = ACK;  
}
```

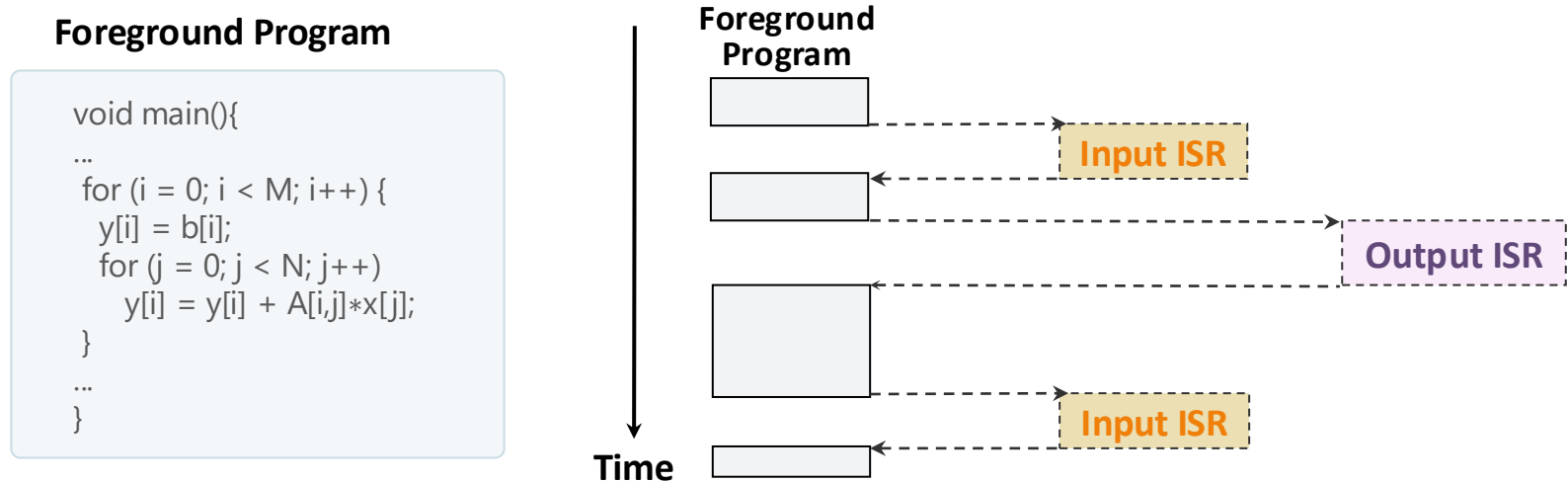
Input ISR adds data and starts output

- Each completion interrupt launches the next queued item.
- The handler disables unnecessary interrupts when no data remains.

```
void output_handler(void) {  
    OUT_STATUS = ACK;  
  
    if (!buffer_empty()) {  
        OUT_DATA = buffer_get();  
        OUT_CONTROL = START;  
    } else {  
        stop_output();  
    }  
}
```

Interrupts (Example) - With Circular Buffer

- The **foreground program** is occasionally interrupted by input and output operations:
 - Handled by the interrupt handlers **in the background!**
- The foreground program resumes after interrupt handlers



Bugs

- The errors can be very hard to find when the interrupt handlers are buggy due to the **concurrency**
- Interrupt handler:
 - must save any CPU register that it will modify
 - must restore them before it exits
- Forgetting to save/restore a register in the handler might cause that register to **mysteriously change a variable** in the foreground program.

Bugs

- For example, in order to set **bit[3]** in word data in address **0x20000000**:

```
...  
LDR R1, =0x20000000 ;Setup address  
STR R2, [R1]         ;Write back 0x79  
...
```

Interrupt!

;Read-Modify-Write Operation

```
LDR  R1, =0x20000000 ;Setup address  
LDR  R0, [R1]         ;Read 0x21
```

Return!

```
ORR.W R0, #0x8        ;Set bit[3]  
STR   R0, [R1]         ;Write back 0x29
```

Bugs

- For example, in order to set **bit[3]** in word data in address **0x20000000**:

```
...  
LDR R1, =0x20000000 ;Setup address  
STR R2,[R1]          ;Write back 0x79  
...
```

Interrupt!

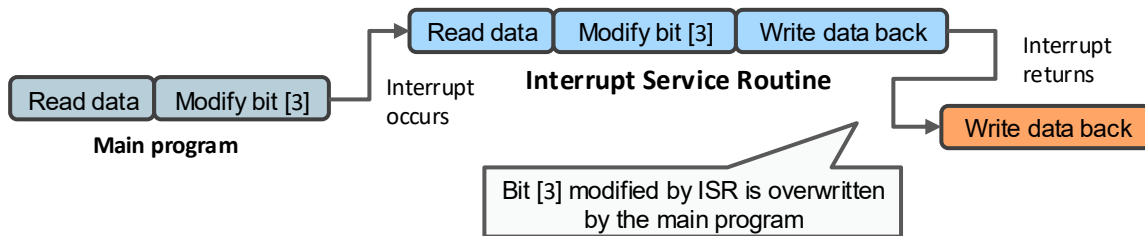
;Read-Modify-Write Operation

```
LDR R1, =0x20000000 ;Setup address  
LDR R0, [R1]         ;Read 0x21
```

Return!

```
ORR.W R0, #0x8       ;Set bit[3]  
STR R0, [R1]         ;Write back 0x29
```

- Read-modify-write operation
 - Reads the data (0x21) from the address 0x20000000
 - The interrupt changes the data of 0x20000000 address Writes back the modified old data back
 - 0x79 has been lost!

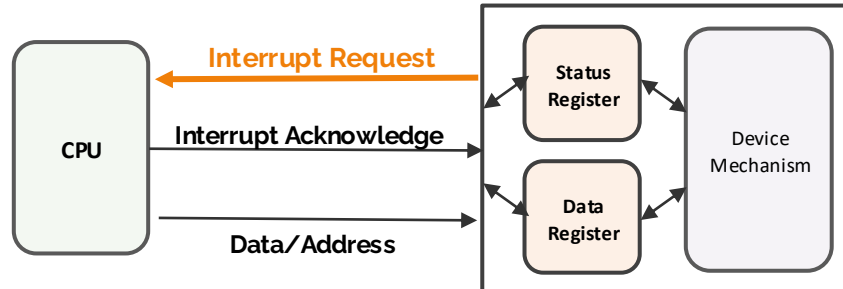


Interrupt handlers should do bounded work

- Capture or deliver the minimum data required by the device
- Clear the interrupt source before it can retrigger unexpectedly
- Avoid blocking calls and unbounded loops
- Move expensive processing into foreground or deferred work
- Measure worst-case execution time

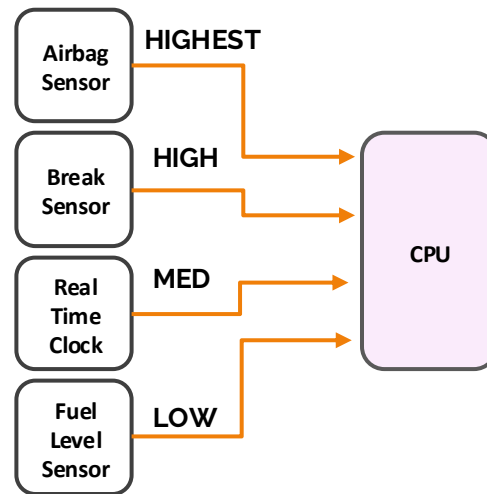
Interrupts - Implementaton

- The CPU checks the **interrupt request (IRQ)** line at every instruction.
- If an interrupt request has been asserted, the CPU:
 - puts the return address on a stack (as CPU does for subroutines)
 - does not fetch the instruction pointed to by the PC
 - sets the PC to the beginning of the **interrupt handler**
- The address of the interrupt handlers are stored **in a table**.



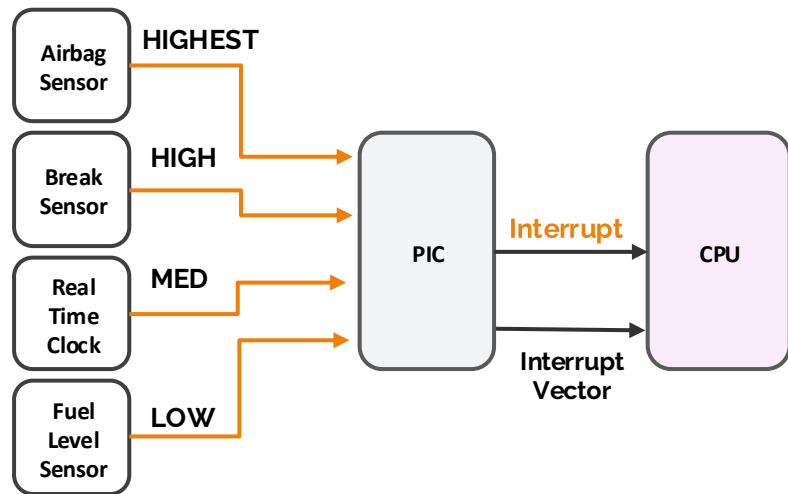
Priorities and Vectors

- Most systems have more than one I/O device:
 - multiple devices can interrupt
 - several interrupt handlers, device addresses
- **Interrupt priorities**
 - recognize some interrupts as more important than others
- **Interrupt vectors**
 - allow the interrupting device to specify its interrupt handler



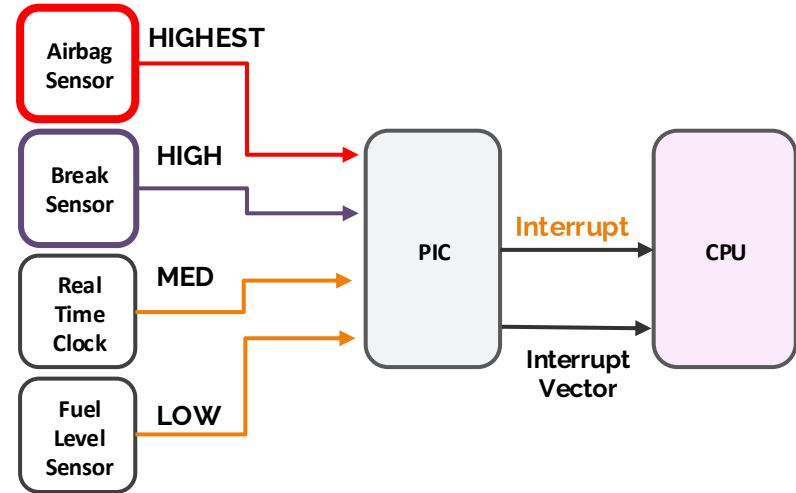
Programmable interrupt controller (PIC)

- **Programmable interrupt controller (PIC)**
 - **Prioritize** multiple interrupt sources so that at any time the **highest priority interrupt** is presented to the core CPU for processing.
- Cortex-M integrates this function in the **NVIC**
 - We will see it later.



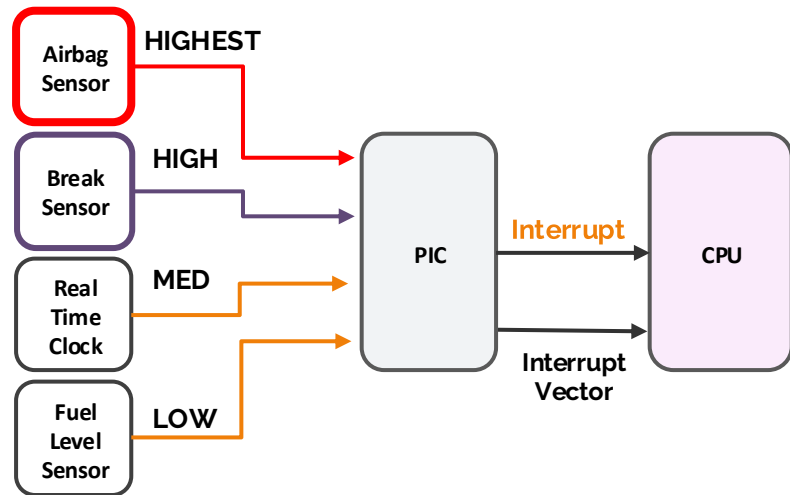
Priorities and Vectors

- **Priority logic** selects the highest eligible request
 - PIC stores the priority level of that interrupt in an internal register
 - When a subsequent interrupt occurs: the priority is checked against current priority
- **Nested interrupts:**
 - a higher priority interrupt source to **preempt** the processing of a lower priority interrupt.



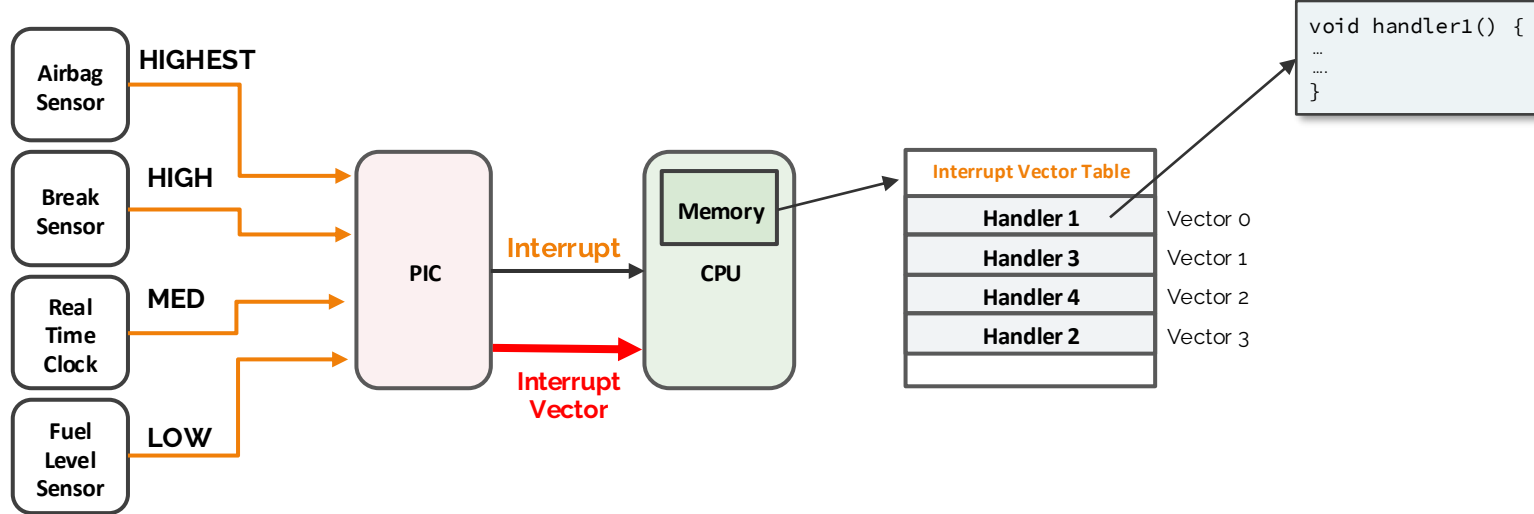
Priorities and Vectors

- **Pending bits** record interrupts awaiting service
- **Masking** interrupts:
 - Enable bits determine which sources may interrupt
 - The highest-priority interrupt is called the **nonmaskable interrupt (NMI)**.
- The NMI **cannot be** turned off
 - e.g., usually reserved for interrupts caused by power failures to save critical state in nonvolatile memory, turn off I/O devices...



Interrupt Vector Table

- Each interrupt has a **vector number**
 - The vector number indexes the vector table
- The selected entry holds the **interrupt vector**:
 - i.e, the memory address of the interrupt handler.



Overhead of Interrupts

- An interrupt causes a change in the program counter:
 - incurs a **branch penalty**
- Interrupt might automatically stores some CPU registers:
 - requires **extra cycles**
- Acknowledge the interrupt and obtain the interrupt vector
 - requires **extra cycles**
- The **interrupt handler overhead**:
 - Saves and restores CPU registers that were not automatically saved
 - The **interrupt return instruction** restores the automatically saved state, incurs a **branch penalty**

Overhead of Interrupts

- **Interrupt Response Time:**

- The time required for the hardware to respond to the interrupt, obtain the vector, save state and so on **cannot be changed by the programmer**.

